

A Depth-Aware, Physics-Based Data Augmentation Framework for Simulating Rain and Fog Distortions in Outdoor-RGB Images

Rongxuan Duan
Pioneer Research Institute

Abstract

1 Introduction

Images used in computer vision are often **distorted**. Distortion in computer vision refers to any signal that produces visual deviation in image/video from its original expectation. When the original **image/video** is unavailable, an observed visual comfort or simply annoying effect on the aesthetic visual appearance is also considered as a distortion [1]. These distortions could be sorted as In-Capture Distortion, including blur, noise, uneven illumination, color shifts, haze, rain, snow, and other visibility changes; and Post-Capture Distortion, such as transmission, impairments, enhancement artefacts, and adversarial distortion.

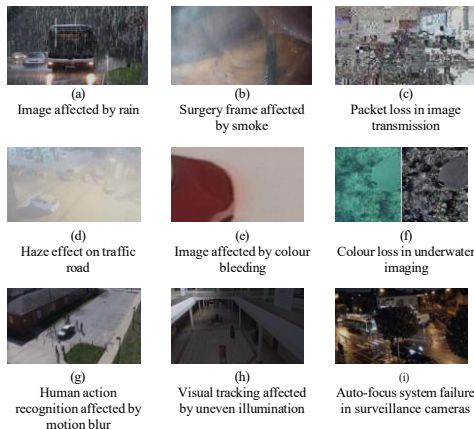


Figure 1: Examples of distortions encountered in computer vision and image/video analysis tasks, adapted from [1]

Although these effects may appear minor to human observers, they can change the statistical structure of an image

and reduce the reliability of machine learning models trained primarily on clean data [1]. Distortions affect the performances of high-level tasks, such as object classification, object detection, and scene understanding [1]. Therefore, distortion detection and removal are a possible preprocessing step for high-level tasks to decrease the impact on performance.

For machine learning, distortion is not only an image-quality issue but **also is affected** by dataset and training. Models usually learn from the distribution of inputs they are given, so a model trained only on clean images may have poor generalizability when working on the same object appearing under poor visibility, weather effects, or degraded contrast. Distortion-specified datasets are important because they make it possible to evaluate model robustness by type and severity, rather than reporting only a single clean-data accuracy score [1].

Current datasets of natural distortions have important limitations. Real rain, haze, fog, and snow images can be difficult to collect at scale, and the resulting data may be unbalanced across weather type, severity, scene category, and annotation quality. Many benchmark datasets also include distortions as broad corruptions rather than as physically meaningful, scene-dependent effects. This limits how well they can support controlled robustness studies or targeted training strategies. The need for environment-aware image formation is also visible in underwater computer vision, where more complicated **distortions present** due to an uneven refractive index of water, such as attenuation, scattering, color-shifting, halo effect, etc. [2].

Related work has explored both distortion recognition and natural-distortion simulation. Survey work on distortion classification emphasizes that multiple and natural distortions remain difficult to categorize and model consistently [1]. In weather-focused vision, rain simulation has been used to render controlled rain effects for robustness evaluation, while haze and fog modeling often relies on at-

Commented [PW1]: David - I am really impressed by the evident in-depth research that you have conducted.

You did a great job of including figures and formulas throughout your paper; this helps to clarify points and connect ideas. I saw that you had concerns about your methodology section, specifically in regards to phrasing, organization, and coherence. I left some comments typically regarding spelling or wording to help you with phrasing and flow. Your organization is logical, and your explanations of the formulas seem coherent. I left other comments throughout the paper for you to consider during your revision process. Keep up the great work!

We look forward to seeing your future drafts and revisions!
- Briana

Commented [PW2]: Spelling:
"distorted"

Commented [PW3]: Spelling:
"image"

Commented [PW4]: Wording:
Consider rewording this to "is also affected" to contribute to greater sentence flow.

Commented [PW5]: Wording:
Consider adding "are" after "distortions" to contribute to greater sentence flow.

atmospheric image-formation ideas such as scattering, attenuation, and reduced scene contrast. Work on adverse-weather perception, including foggy driving scenes, also shows that real weather can strongly affect downstream recognition and detection performance.

The model trains a distortion augmentation framework, which adds controlled rain and fog effects to existing baseline datasets such as BDD100K, ACDC and Cityscapes [3, 4, 5]. The framework is evaluated by comparing the performances of a downstream object detection model, YOLOv8n [6, 7], on the original dataset and the augmented dataset.

1.1 Cycle-GAN (Generative Adversarial Network)

Generative Adversarial Network (GAN) is a deep learning model that consists of two networks that compete against each other: a generator G and a discriminator D . The generator G tries to generate realistic images from a target domain X , while the discriminator D tries to discriminate generated images from real images. That is, the generator learns a mapping: $G : X \rightarrow Y$. For example, in the context of our work, the generator G tries to create rainy or foggy looking images from clear images, while the discriminator D tries to distinguish between real rainy or foggy images and generated images. The generator and discriminator are trained in an adversarial manner, where the adversarial framework's objective is to minimize the generator loss and maximize the discriminator loss. The loss function of GAN is given below, defined by Goodfellow et al[8].

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]. \quad (1)$$

Traditional GANs use paired training data, which consists of training examples $\{x_i, y_i\}_{i=1}^N$, where x_i is the input and y_i is the corresponding output. GAN is useful for image-to-image translation tasks, such as photo reconstruction from edge maps, colorization, photo synthesis from label maps, etc[9]. However, in many cases, paired training data is not available. In the context of our work, it is difficult to collect paired training data of clear images and corresponding rainy or foggy images at same places. Existing datasets, such as BDD100K, cityscapes, or ACDC, are all unpaired datasets[3, 5, 4]. Figure 2 illustrates the difference between paired and unpaired training data.

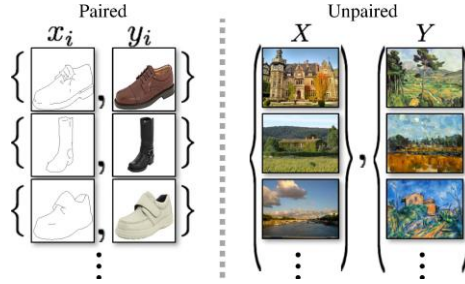


Figure 2: Paired training data (left) consists of training examples $\{x_i, y_i\}_{i=1}^N$. Unpaired training data (right) consists of a source set $\{x_i\}_{i=1}^N$ ($x_i \in X$) and a target set $\{y_i\}_{i=1}^N$ ($y_i \in Y$). Unpaired training contains no correspondence from the source set X to the target set Y . Figure adapted from [10].

Our work uses Cycle-GAN, which is a type of GAN that learns from unpaired training data. Cycle-GAN uses two generators and two discriminators. A generator $G : X \rightarrow Y$ transforms images from domain X to domain Y ; a reverse generator $F : Y \rightarrow X$ transforms images from domain Y to domain X . The first discriminator D_Y distinguishes between real images in domain Y and generated images from generator G ; the second discriminator D_X distinguishes between real images in domain X and generated images from generator F . The Cycle-GAN framework uses a cycle-consistency loss to ensure that the generated images can be transformed back to the original domain. The cycle-consistency loss is given below, defined by Zhu et al[10].

$$\mathcal{L}_{\text{cyc}}(G, F) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_{\text{data}}(y)} [\|G(F(y)) - y\|_1]. \quad (2)$$

The objective of Cycle-GAN is to minimize the adversarial loss, maximize the discriminator loss, and minimize the cycle-consistency loss. The key idea for Cycle-GAN is that if an image is transformed from domain X to domain Y by G , and then it is transformed back to domain X by F , the resulting image should be similar to the original image. This cycle-consistency allows the model to not only produces an image that looks like it belongs to the target domain, but also preserves the content, or the structure of the original image[10].

In weather augmentation, cycle-GAN is useful because it generates realistic rainy or foggy images from clear images while it preserves the scene structures and it does not require paired training data. However, a limitation of Cycle-GAN, as well as other GANs, is that it is not controllable. It is impossible to control how much wetness, rain, or fog is gen-

Commented [PW6]: Spacing:
Consider placing a space between these two words.

Commented [PW9]: Spelling:
"domain"

Commented [PW7]: Spelling:
"paired"

Commented [PW8]: Wording:
Consider adding "the" after "at" to contribute to greater sentence flow.

Commented [PW10]: Commas:
Consider placing a comma after "structures" to indicate the separation between the two independent clauses.

erated, and it is impossible to control the depth-dependent effects of rain and fog [10, 11]. Therefore, our work uses Cycle-GAN to generate rainy and foggy images, but it also uses a physics-based, depth-aware renderer to generate rain with controllable appearance.

2 Related Work

2.1 Distortion Classification and Evaluation

Traditional distortion classification models and evaluation use blur indices, noise estimators, natural scene statistics, image quality metrics, Support Vector Machines (SVM), and K-Nearest Neighbours (KNN) to evaluate the impact of distortions. Recent distortion classification and detection models use Convolutional Neural Networks (CNN) and temporal models. Ahn, Kang, and Sohn train a Y-shaped CNN for joint distortion classification and localization. Other methods for distortion classification include feature pool, relating metrics with specific distortions, and Natural Scene Statistics (NSS). The method supports learned distortion features, yet most datasets focus on controlled distortions rather than natural weather [1, 12].

The quality of visual content in images and distortion severity are important factors that affect performance in high-level tasks, including object classification, object detection, and scene understanding. Using distortion classification and detection models could help researchers finetune models that are more resistant to distortions. However, our work helps finetuning models by presenting a data augmentation framework that increases models' resilience to rain and fog distortions from training, which does not require distortion detection and distortion removal process.

2.2 Current Limitations of Distortion Datasets

Current distortion datasets give labels and severity levels, but many examples come from synthetic, single-type, quality-assessment settings. LIVE, TID2008, TID2013, CSIQ, Waterloo, KADID-10K, LIVEMD, and MDID help keep a controlled evaluation. However, they do not contain natural weather with precise physical labels. Rain, haze, fog, and snow depend on lighting, exposure, particle density, scene depth, and occlusion. There is a lack of labeled, controlled multi-distortion datasets. Consequently, classification and localization of multiple distortions remains challenging due to lack of labeled, controlled datasets [1, 11]. Our work adds controlled, artificial distortions, which addresses lack of labeled, controlled datasets, especially for outdoor datasets.

2.3 Traditional Rain Rendering

Garg and Nayar model rain streak photometry and spatio-temporal behavior. Their work links rain visibility to drop size, velocity, exposure time, scene radiance, camera settings, and drop shape oscillation. This result matters for rendering because a transparent 2D texture ignores camera and scene factors. Traditional rain rendering lacks the effect of moisture on the scene, and it often appears to be unrealistic. For instance, traditional rain rendering lacks wet surfaces such as water on the ground, and it creates raindrops that are supposed to be hidden behind buildings [11]. Though, this framework supported rain detection, removal, rendering, and camera-based rain measurement [13].

Below exemplifies three traditional rain rendering methods: rain100H, rain800, and did-MDN [11].

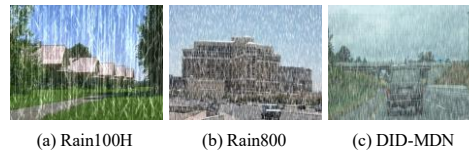


Figure 3: Three traditional rain rendering methods, adapted from [11]

2.4 Modern Rain Rendering

Modern rain rendering turns rain models into dataset augmentation for robustness tests. Halder, Lalonde, and de Charette combine a particle simulator, outdoor illumination estimation, and rain photometry to render rain and fog over clear driving images. The pipeline preserves labels from KITTI and Cityscapes while changing weather appearance. Detection and segmentation results then measure model sensitivity under controlled rain [14].

Tremblay, Halder, de Charette, and Lalonde compare three rendering routes: physics-based, data-driven, and hybrid. They classify raindrops into rain particles and fog-like rain. Fog-like rain is a single pixel that contains multiple rain particles; it is located far away from the camera and appears as fog. The physics-based route combines fog-like rain, rain particles, and photometric modeling. The renderer estimates outdoor illumination because a raindrop receives light from a wider angular range (approximately 160 degrees) than the camera's view (70 to 100 degrees). This design improves physical control over a flat overlay. The same design also depends on sky visibility, environmental symmetry about photometry, and compute cost [11, 15, 16].

The data-driven route uses a GAN-style image translation model. A GAN trains two networks: a generator synthesizes target-domain images, and a discriminator judges whether each image looks real or generated. Adversarial

Commented [PW13]: Spelling: "exemplifies"

Commented [PW11]: Spacing: Consider placing a space between these two words.

Commented [PW12]: Noun-Verb Agreement: Consider changing "remains" to "remain" to match the plurality of the noun.

training on the generator outputs rainy-looking images. The output from the generator contains features such as dark sky, rainy environment, wet surfaces and dim lamination. However, GAN is less controlled, which risks rain density, depth ordering, and scene consistency. GAN's focus on global appearance also creates unrealistic features, such as fake water reflections [11, 9, 17].

Tremblay et al. render rain over KITTI, Cityscapes, and nuScenes. They evaluate object detection, semantic segmentation, and depth estimation under increasing degradation. The paper reports degrading performances as increasing rain intensity and more robust performances after finetuning using rain-augmented datasets. The paper reports that outputs from GAN only are user-rated as the most realistic images; outputs from GAN plus PBR perform the best on data augmentation [11].

Our work is inspired from the framework from Tremblay et al. our work uses the traditional rain rendering framework with depth estimation to build rain streaks with controlled, depth-dependent rain streaks; our work also involves using a Cycle-GAN model that adds rainy and foggy visual effects.

2.5 Fog Rendering

Fog rendering often follows the atmospheric image formation model. Scene radiance attenuates with distance, while airlight grows along the camera's ray. Sakaridis, Dai, and Van Gool applies this model to clear Cityscapes images and produce Foggy Cityscapes. Depth makes fog stronger for far regions rather than uniform across the image. Training with synthetic fog improves semantic foggy scene understanding, but results depend on depth quality, density choices, and the synthetic-to-real gap [18].

Clear2Fog extends this direction with monocular depth estimation and atmospheric light estimation. Monocular depth estimation does not require 3D constructions such as LiDAR; rather, this paper uses Apple Depth Pro to produce an estimated depth map. The pipeline simulates fog on clear-weather driving datasets and tries to preserve camera-LiDAR consistency. Without relying on LiDAR, Clear2Fog remains stable under extreme conditions like fog, snow or haze. Seeing Through Fog Without Seeing Fog focuses on multimodal perception rather than rendering. The dataset and fusion model show different failure patterns across camera, LiDAR, radar, and gated near-infrared sensors [19, 20].

2.6 Depth Estimation and Application in Distortion Rendering

Depth provides the geometry needed for natural distortion placement. In haze and fog, depth sets attenuation and airlight strength. In rain, depth helps determine the severity, intensity and placement of rain streaks. A renderer without

geometry often places plausible-looking weather in physically inconsistent locations [18, 11].

Depth estimation models estimate distance from an image or complete sparse sensor depth. The output supplies an approximate geometry map when a dataset lacks dense ground truth. Jaritz et al. study sparse and dense depth completion for driving scenes. Depth Anything V2 offers a modern monocular option trained with synthetic images and pseudo-labeled real images. This makes Depth Anything V2 useful for RGB-only distortion rendering [21, 22].

Our work uses the depth-map output from Depth Anything V2 as the guiding parameter for transparency and appearance of rain streaks and translucency and intensity of fog. The depth-map output from Depth Anything V2 is stable and reliable, and it is a strong model for completing our computer vision tasks.

2.7 Datasets With Distortions

General distortion and corruption datasets support baselines, but natural weather rendering requires more geometry and scene context. The distortion survey lists LIVE, TID2008, TID2013, CSIQ, Waterloo, KADID-10K, LIVEMD, and MDID as major image-quality or multi-distortion resources. ImageNet-C standardizes common corruption robustness for image classifiers. These resources help robustness framing. They do not model depth-aware rain, haze, or fog. [1].

Weather-specific datasets match the project more closely.

ACDC includes fog, nighttime, rain, and snow with dense driving-scene annotations and normal-condition correspondences for many scenes. Foggy Cityscapes and rain-augmented KITTI, Cityscapes, and nuScenes preserve existing labels while adding synthetic weather. The limitation of these datasets is the lack of controlled severity or intensity, depth-aware labels, and controlled natural distortions [4, 18, 11].

2.8 BDD100K

BDD100K serves as a baseline dataset for our work, because the data are large, diverse, and driving-focused. The dataset contains 100,000 driving videos and supports detection, segmentation, lane marking, drivable area, and related driving labels. BDD100K also includes geographic, environmental, and weather variation. The limitation is missing paired clean/adverse views and controlled severity labels. A depth-aware rain and haze renderer would turn BDD100K into a controlled robustness testbed while keeping real driving diversity. [23].

Commented [PW15]: Spelling:
"Weather-specific"

Commented [PW14]: Noun-Verb Agreement:
Consider changing this to "apply" to match the plurality of the noun.

Commented [PW16]: Punctuation:
Consider removing this period, as the period following the citation is sufficient.



Figure 4: Examples of clear, rainy, and foggy images from BDD100K [23]

3 Methodology

Data augmentation is a strategy for improving computer vision models by artificially increasing the diversity of training images through transformations such as cropping, rotating, flipping and adding noise. These methods are especially useful when access to labeled image data is limited, because they expose a model to variations it may encounter in real-world visual tasks. In computer vision, data augmentation helps image classifiers learn features that are less dependent on a single viewpoint, orientation, or image style, reducing overfitting and improving generalization[26].

Data augmentation is the primary method to increase a dataset's size and variety. In the context of this paper, it refers to adding rain and fog to images. As discussed in previous sections, collecting data with various natural distortions for an outdoor dataset is challenging. Therefore, using data augmentation to render natural distortions improves the performance for deep learning models. Our work focuses on rain and fog distortions in outdoor RGB images.

3.1 Rain Rendering

Rain appears as dynamic-looking particles and brings a rainy effect to the image. Inspired by Tremblay et al [11], our rain rendering method uses a physics-based rain rendering method that simulates raindrops and a Cycle-GAN model that creates rainy effect.

3.1.1 Physics-based Rendering (PRB)

To generate synthetic raindrop appearance, the physics-based rain rendering (PBR) method combines image-rspace rain streak synthesis, atmospheric scattering and linear-light compositing, based on the monocular depth map produced by Depth Anything V2[22]. The objective of this renderer is to produce controllable raindrops while being consistent with scene geometry and depth.

Given an input RGB/BGR image I and an estimated depth map D , the raw depth values are first normalized into a relative distance map:

$$D_n = \frac{\text{clip}(D, p_1, p_{99}) - \min(D)}{\max(D) - \min(D) + \epsilon} \quad (3)$$

where p_1 and p_{99} are the 1st and 99th percentiles of the depth values used to exclude extreme values, and ϵ is a small constant to avoid division by zero. The normalized scene distance is then defined as $d(x, y) = 1 - D_n(x, y)$, where a $d(x, y) = 0$ represents foreground regions close to the camera and $d(x, y) = 1$ represents distant scene regions.

The renderer models rain as layered rain streaks. Four rain-depth layers are used: near, mid, far, and background. Each layer l has a fixed normalized depth z_l , a streak count N_l , streak length L_l , streak width W_l , base opacity a_l , defocus blur, and angular jitter. For an image of height H , width W , minimum dimension $m = \min(H, W)$, area scale $A = HW/1000$, and rain severity s . The layer parameters are defined as functions of severity. For example, the near layer uses:

$$\begin{aligned} N_{\text{near}} &= A(0.025 + 0.13s) \\ L_{\text{near}} &= m(0.025 + 0.080s) \\ W_{\text{near}} &= 1.75 + 2.20s \\ a_{\text{near}} &= 0.020 + 0.055s \end{aligned}$$

The mid, far, and background layers follow the same structure, with progressively smaller streak widths and lengths but larger particle counts. This models the physical appearance of raindrops, which appear to be smaller and more numerous as they are positioned further from an observer. Nearby raindrops appear longer, thicker and more defocused, while distant raindrops appear denser and smaller.

Depth is used to determine whether rain in a given layer should be visible at each pixel. A rain layer is only visible when it lies in front of the visible scene surface. Raindrops from a given layer do not suddenly appear or disappear; rather, they fade in or out when their depth $d(x, y)$ is at a depth-edge, where it is close to a layer's fixed normalized depth z_l . The visibility is computed using a smooth transition function:

$$V_l(x, y) = \text{smoothstep}(-\delta_l, \delta_l, d(x, y) - z_l) \quad (4)$$

where δ_l is the layer softness parameter that controls how rapidly the raindrops fade in or out. A smaller δ_l means a sharper transition, while a larger δ_l means a smoother transition. The near layer has the smallest δ_l , while the background layer has the largest δ_l . The smoothstep function is:

$$\begin{aligned} \text{smoothstep}(a, b, x) &= t^2(3 - 2t) \\ t &= \text{clip}\left(\frac{x - a}{b - a + \epsilon}, 0, 1\right) \end{aligned}$$

This prevents distant rain from being visible in front of foreground objects. The smoothstep function and visibility

Commented [PW21]: Spelling: "fixed"

Commented [PW22]: Spelling: "streak"

Commented [PW17]: Spacing: Consider adding a space before the beginning bracket to ensure consistency of your citation format throughout your paper.

Commented [PW23]: Spelling: "lengths"

Commented [PW18]: Wording: Consider replacing "for" with "of" to contribute to greater sentence flow.

Commented [PW24]: Spelling: "disappear"

Commented [PW19]: Wording: Consider changing this to "effects."

Commented [PW20]: Spelling: "normalized"

function prevents discontinuities in the rain appearance at a depth-edge. It also prevents halo spaces around object boundaries.

Individual rain streaks are generated using an anisotropic motion kernel, which is a kernel that blurs differently in different directions, rather than hard lines. For a streak angle θ , each kernel pixel is projected into coordinates along and across the rain direction:

$$\begin{aligned} u &= x \cos \theta + y \sin \theta \\ v &= -x \sin \theta + y \cos \theta \end{aligned}$$

where u is distance along the rain streak and v is distance across the rain streak.

The streak intensity (how strong each pixel of a rain streak is inside the filter) is tapered along its length and Gaussian blurred along its width:

$$K(u, v) = \max \left(0, 1 - \frac{|u|}{L/2} \right)^p \cdot \exp \left(-\frac{1}{2} \frac{v^2}{\sigma_w} \right) \quad (5)$$

where $p = 1.45$ controls tapering and:

$$\sigma_w = \max(0.35, 0.55W)$$

The kernel is normalized so its peak value is one. By applying this kernel on point-like rain particle, it creates motional rain streaks that resembles real rain's photometry rather than opaque white streaks.

For each sampled rain streak, the opacity α_i is defined by a log-normal distribution and the layer opacity:

$$\alpha_i = \text{clip}(\alpha_l \cdot \text{LogNormal}(0, 0.22), 0.001, 0.18)$$

The brightness level Y of a streak is made dependent on the local scene brightness. The local scene brightness is obtained from converting the image to linear light:

$$Y = 0.0722B + 0.7152G + 0.2126R$$

The rain streak contrast factor controls how visible a rain streak should be depending on local brightness. It is stronger over dark regions and weaker over bright regions:

$$C = 0.34 + 1.05(1 - Y)^{0.72}$$

The pixel intensity delta for a rain streak is its change in brightness compared to the scene brightness Y :

$$\Delta_i = \epsilon_l C - \eta_l Y^{1.15} + \mathbf{N}(0, 4 + 5\alpha_l)$$

where ϵ_l is the layer highlight strength constant and η_l is a shadow suppression constant. Delta is then clipped to a range $\text{clip}(\Delta_i, -16, 135)$.

The renderer then accumulates rain as an optical thickness map. Each raindrop's pixel intensity Δ_i , convolved

with the motion kernel, is added to the map. The optical thickness map is denoted as $\tau(x, y)$. Then, the renderer generates an opacity-weighted brightness map as:

$$\tau \Delta(x, y) = \sum_i \tau(x, y) \Delta_i$$

The final rain alpha is defined as:

$$\alpha_{rain} = \text{clip}(1 - e^{-\text{clip}(\tau, 0, 3.5)}, 0, 0.62)$$

The final rain color delta map Δ_{rain} is the opacity-weighted average and clipped within the plausible range $(-18, 145)$.

A light mist layer is applied before compositing the streaks, based on the depth map. The mist intensity depends on layer, where a more distant layer has a higher intensity, a smoothstep function is applied, preventing abrupt change in mist appearance between across layers. The mist map is denoted as I_{mist} , and it is then added with the slightly tinted rain delta map Δ_{rain} to produce a weather augmentation mask, I_{target} .

The output image is then:

$$I_{out} = I_{mist}(1 - \alpha_{rain}) + I_{target}\alpha_{rain} \quad (6)$$

Overall, the renderer combines depth estimation with physics-based optical accumulation. The depth-based layer visibility prevents rain from incorrectly appearing over foreground objects. the optical thickness map τ blends the opacities of rain streaks that are co-linear in a line from the observer, rather than overlaying rain streaks directly onto the input image, causing a local-unrealistic exposure. The brightness-aware streaks ensure the brightness to be adjustable depending on the background and depth.

Commented [PW25]: Spelling: "generated"

Commented [PW26]: Wording: Consider replacing "a" with "an," as the following word begins with a vowel sound.

Commented [PW27]: Commas: Consider placing a comma after "normalized" to indicate the separation between the two independent clauses.

Commented [PW28]: Spelling: "visible"



Clear weather input Depth-aware rain augmented output

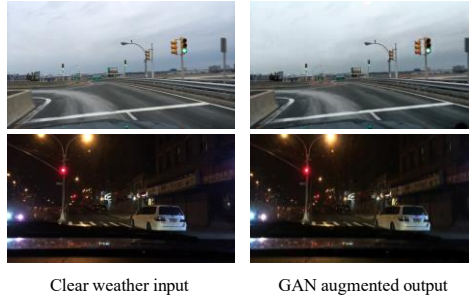
Figure 5: Images in the left column are the clear-weather-labeled inputs from BDD100K dataset. Images in the right column are the corresponding depth-aware rain augmented output, severity parameter being set as "medium".

3.1.2 Rain Cycle-GAN

While the physics-based renderer simulates raindrops and fine mist, it lacks rainy characteristics such as wetness, reflections, sky and lighting. Scenes generated from the physics-based render may have realistic raindrops and mist, but they lack an overall rainy look. These characteristics are hard to generate through a controlled method, whereas Cycle-GAN can handle image-to-image translation tasks.

The dataset used for training is BDD100K. It contains seven labels: "clear", "overcast", "undefined", "snowy", "rainy", "partly cloudy" and "foggy". The corresponding numbers of images in train split are 37344, 8700, 8119, 5549, 5070, 5881, 130, respectively; the corresponding numbers of images in validation split are 5346, 1239, 1157, 769, 738, 738, 13, respectively. Labels "clear" and "rainy" are used for training, which refer to clear weather and rainy weather, respectively.

For an unpaired Cycle-GAN training, the training process loads 3000 clear images and 3000 rainy images. Training images are loaded as 512*512, then they are randomly cropped at 384*384. The model is optimized with 10 epochs and 10 decaying epochs, learning rate of 0.0001 and batch size of 1.



Clear weather input GAN augmented output

Figure 6: Images in the left column are the clear-weather-labeled inputs from BDD100K dataset. Images in the right column are the corresponding Rain Cycle=GAN augmented output.

The generated images have more foggy and desaturated skies and wet effects on the ground in common. However, some of them have random white noise. This issue is common when the size of training images is too small, or the size of the train split is small. It could be fixed by increasing image resolution or using a bigger training dataset.

A large portion of the rainy images in BDD100K appear to have been captured after rainfall rather than during active rain. This may explain why the differences between the GAN-generated outputs and the input images are subtle and not always noticeable to the naked eye.

3.1.3 Framework Combining PBR and Rain Cycle-GAN

The final rain augmentation framework combines the Rain Cycle-GAN model and the depth-aware physics-based renderer into a hybrid pipeline. The purpose is to use the strengths of both methods. Rain Cycle-GAN generates the overall rainy appearance, while the physics-based rain renderer adds controllable rainfall, including rain streaks, fine mist, and depth-aware visibility.

The pipeline starts with a clear-weather image from BDD100K. First, the image is passed through the trained Rain Cycle-GAN generator to produce a rainy-looking output of the same scene. The output preserves structures such as road layout, vehicles, buildings, etc; it shifts the image toward the rainy domain. Depth Anything V2 also estimates a monocular depth map from the input image, which is used by the PBR renderer.

After the Cycle-GAN translation, the PBR renderer composites rain onto the GAN-generated image. The severity parameter controls the number and appearance of rain streaks. Near rain layers contain larger and more visible streaks, while far layers contain smaller and denser streaks. The

Commented [PW33]: Wording:
Consider adding "the" after "from" to contribute to greater sentence flow.

Commented [PW29]: Wording:
Consider adding "the" after "from" to contribute to greater sentence flow.

Commented [PW30]: Punctuation:
Consider placing a period at the end of this sentence.

Commented [PW31]: Wording:
Consider removing "an" to contribute to greater sentence flow.

Commented [PW34]: Spelling:
"preserves"

Commented [PW35]: Spelling:
"estimates"

Commented [PW32]: Wording:
Consider adding "a" before "learning" and before "batch."

depth map prevents inconsistent results, such as distant rain appearing in front of nearby cars or buildings.

The order is important. If the physics-based rain is added before Cycle-GAN, the generator may weaken or distort the synthetic rain streaks. By applying PBR after Cycle-GAN, the result may consistently keep the global rainy style while adding controllable rain.

Compared with Cycle-GAN alone, the hybrid framework provides direct control over rain intensity and depth placement. Compared with PBR alone, it produces a more complete rainy scene because it includes wetness and lighting changes. Therefore, the combined PBR and Rain Cycle-GAN framework assembles real rainy images better than PBR or GAN alone, and it can produce controlled rainy training data with smaller visual gap between synthetic rain and real rainy scenes.

References

- [1] Muhammad Ali Qureshi et al. *Distortion Classification in Computer Vision Applications: Current Progress, Challenges, and Perspectives — ACM Computing Surveys*. <https://dl.acm.org/doi/10.1145/3773023>. (Visited on 06/10/2026).
- [2] A Survey on Underwater Computer Vision — *ACM Computing Surveys*. <https://dl.acm.org/doi/10.1145/3578516>. (Visited on 06/10/2026).
- [3] C. K. Wolfe. *BDD100K: A Large-scale Diverse Driving Video Database*. (Visited on 06/14/2026).
- [4] Christos Sakaridis et al. *ACDC: The Adverse Conditions Dataset with Correspondences for Robust Semantic Driving Scene Perception*. <https://arxiv.org/abs/2104.13395>. Apr. 2021. (Visited on 06/24/2026).
- [5] Marius Cordts et al. *The Cityscapes Dataset for Semantic Urban Scene Understanding*. Apr. 2016. DOI: 10.48550/arXiv.1604.01685. arXiv: 1604.01685 [cs.CV]. (Visited on 07/02/2026).
- [6] What Is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector. <https://arxiv.org/html/2408.15857v1>. (Visited on 07/03/2026).
- [7] Explore Ultralytics YOLOv8. <https://docs.ultralytics.com/models/yolov8>. Nov. 2023. (Visited on 07/03/2026).
- [8] Ian J Goodfellow et al. “Generative Adversarial Nets”. In: ().
- [9] Phillip Isola et al. *Image-to-Image Translation with Conditional Adversarial Networks*. Nov. 2018. DOI: 10.48550/arXiv.1611.07004. arXiv: 1611.07004 [cs.CV]. (Visited on 06/16/2026).
- [10] Jun-Yan Zhu et al. *Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks*. Aug. 2020. DOI: 10.48550/arXiv.1703.10593. arXiv: 1703.10593 [cs.CV]. (Visited on 06/30/2026).
- [11] Maxime Tremblay et al. “Rain Rendering for Evaluating and Improving Robustness to Bad Weather”. In: *International Journal of Computer Vision* 129.2 (Feb. 2021), pp. 341–360. ISSN: 1573-1405. DOI: 10.1007/s11263-020-01366-3. (Visited on 06/10/2026).
- [12] Namhyuk Ahn, Byungkoo Kang, and Kyung-Ah Sohn. “Image Distortion Detection Using Convolutional Neural Network”. In: *2017 4th IAPR Asian Conference on Pattern Recognition (ACPR)*. Nanjing, China: IEEE, Nov. 2017, pp. 220–225. ISBN: 978-1-5386-3354-0. DOI: 10.1109/ACPR.2017.95. (Visited on 06/10/2026).
- [13] Kshitiz Garg and Shree K. Nayar. “Vision and Rain”. In: *International Journal of Computer Vision* 75.1 (Oct. 2007), pp. 3–27. ISSN: 1573-1405. DOI: 10.1007/s11263-006-0028-6. (Visited on 06/14/2026).
- [14] Shirsendu Sukanta Halder, Jean-François Lalonde, and Raoul de Charette. *Physics-Based Rendering for Improving Robustness to Rain*. Aug. 2019. DOI: 10.48550/arXiv.1908.10335. arXiv: 1908.10335 [cs.CV]. (Visited on 06/14/2026).
- [15] Yannick Hold-Geoffroy et al. *Deep Outdoor Illumination Estimation*. Apr. 2018. DOI: 10.48550/arXiv.1611.06403. arXiv: 1611.06403 [cs.CV]. (Visited on 06/14/2026).
- [16] Jinsong Zhang et al. *All-Weather Deep Outdoor Lighting Estimation*. June 2019. DOI: 10.48550/arXiv.1906.04909. arXiv: 1906.04909 [cs.CV]. (Visited on 06/16/2026).
- [17] Xun Huang et al. “Multimodal Unsupervised Image-to-Image Translation”. In: *Computer Vision – ECCV 2018*. Ed. by Vittorio Ferrari et al. Cham: Springer International Publishing, 2018, pp. 179–196. ISBN: 978-3-030-01219-9. DOI: 10.1007/978-3-030-01219-9_11.
- [18] Christos Sakaridis, Dengxin Dai, and Luc Van Gool. “Semantic Foggy Scene Understanding with Synthetic Data”. In: *International Journal of Computer Vision* 126.9 (Sept. 2018), pp. 973–992. ISSN: 1573-1405. DOI: 10.1007/s11263-018-1072-8. (Visited on 06/14/2026).

Commented [PW36]: Spelling: "rain"

Commented [PW37]: Wording: Consider changing "apply" to "applying" to provide greater clarity.

Commented [PW38]: Wording: Consider adding "a" after "with" to contribute to greater sentence flow.

- [19] Mohamed Ahmed Mohamed and Xiaowei Huang. *A Data Efficiency Study of Synthetic Fog for Object Detection Using the Clear2Fog Pipeline*. May 2026. DOI: 10.48550/arXiv.2605.12608. arXiv: 2605.12608 [cs.CV]. (Visited on 06/18/2026).
- [20] Mario Bijelic et al. "Seeing Through Fog Without Seeing Fog: Deep Multimodal Sensor Fusion in Unseen Adverse Weather". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2020, pp. 11682–11692. DOI: 10.1109/CVPR42600.2020.01170.
- [21] Maximilian Jaritz et al. *Sparse and Dense Data with CNNs: Depth Completion and Semantic Segmentation*. Aug. 2018. DOI: 10.48550/arXiv.1808.00769. arXiv: 1808.00769 [cs.CV]. (Visited on 06/14/2026).
- [22] Lihe Yang et al. *Depth Anything V2*. <https://arxiv.org/abs/2406.09414v2>. June 2024. (Visited on 06/24/2026).
- [23] Fisher Yu et al. *BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning*. <https://arxiv.org/abs/1805.04687v2>. May 2018. (Visited on 06/24/2026).
- [24] Md. Imtiyaz Anwar and Arun Khosla. "Vision Enhancement through Single Image Fog Removal". In: *Engineering Science and Technology, an International Journal* 20.3 (June 2017), pp. 1075–1083. ISSN: 22150986. DOI: 10.1016/j.jestech.2016.11.015. (Visited on 06/18/2026).
- [25] Mohamed Ahmed Mohamed and Xiaowei Huang. *A Data Efficiency Study of Synthetic Fog for Object Detection Using the Clear2Fog Pipeline*. <https://arxiv.org/abs/2605.12608v1>. May 2026. (Visited on 06/24/2026).
- [26] Luis Perez and Jason Wang. *The Effectiveness of Data Augmentation in Image Classification Using Deep Learning*. Dec. 2017. DOI: 10.48550/arXiv.1712.04621. arXiv: 1712.04621 [cs.CV]. (Visited on 07/05/2026).